Pattern Matching Expressions in C#

🔷 1. What is Pattern Matching?

Pattern Matching in C# allows you to:
👉 Check a value
👉 Extract data from it
👉 Apply conditions

🔷 2. Why Pattern Matching?

Traditional code:

- Verbose
- Error-prone (casting issues)
- Hard to read

Pattern matching:

- Cleaner
- Safer (no invalid cast exceptions)
- More expressive

🔷 3. Types of Pattern Matching in C#

We'll cover all major types:

1. **Type Pattern**
2. **Constant Pattern**
3. **Relational Pattern**
4. **Logical Pattern**
5. **Property Pattern**
6. **Positional Pattern**

7. **Var Pattern**
8. **Discard Pattern**

🔷 4. Type Pattern

Used to check type and assign variable.

```
object obj = "Hello";

if (obj is string text)
{
    Console.WriteLine(text.Length);
}
```

👉 text is available only if match succeeds

🔷 5. Constant Pattern

Match against a fixed value.

```
int number = 10;

if (number is 10)
{
    Console.WriteLine("Number is 10");
}
```

Also useful in switch:

```
int day = 1;

string result = day switch
{
```

```
    1 => "Monday",
    2 => "Tuesday",
    _ => "Unknown"
};
```

🔷 6. Relational Pattern (C# 9+)

Used for comparisons.

```
int age = 25;

if (age is > 18)
{
    Console.WriteLine("Adult");
}
```

🔷 7. Logical Patterns (AND, OR, NOT)

Combine conditions.

```
int age = 25;

if (age is > 18 and < 60)
{
    Console.WriteLine("Working age");
}
```

OR example:

```
if (age is < 18 or > 60)
{
    Console.WriteLine("Dependent");
```

```
}
```

NOT example:

```
if (age is not 18)
{
    Console.WriteLine("Not exactly 18");
}
```

🔷 8. Property Pattern

Used to match object properties.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

Person p = new Person { Name = "John", Age = 30 };

if (p is { Age: > 18 })
{
    Console.WriteLine("Adult");
}
```

🔷 9. Nested Property Pattern

```
class Address
{
    public string City { get; set; }
```

```
}

class Person
{
    public Address Address { get; set; }
}

if (p is { Address: { City: "Hyderabad" } })
{
    Console.WriteLine("From Hyderabad");
}
```

🔷 10. Positional Pattern

Works with **deconstruct()** method.

```
class Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y) => (X, Y) = (x, y);

    public void Deconstruct(out int x, out int y)
    {
        x = X;
        y = Y;
    }
```

```
}

Point pt = new Point(10, 20);

if (pt is (10, 20))
{
    Console.WriteLine("Matched Point");
}
```

🔷 11. Var Pattern

Always matches and assigns.

```
int number = 5;

if (number is var x)
{
    Console.WriteLine(x); // Always executes
}
```

👉 Useful in expressions

🔷 12. Discard Pattern

Ignore value using _

```
if (obj is _)
{
    Console.WriteLine("Always true if not null");
}
```

🔷 13. Switch with Pattern Matching (Most Powerful)

```
object input = 25;


string result = input switch
{
    int n when n > 0 => "Positive integer",
    string s => $"String of length {s.Length}",
    null => "Null value",
    _ => "Unknown"
};
```

👉 This is heavily used in:

- APIs
- Business logic
- Validation layers

🔷 14. Real-Time Example (API Scenario)

```
public string GetDiscount(object customer)
{
    return customer switch
    {
        PremiumCustomer pc when pc.Purchases > 10000 => "20%",
        PremiumCustomer => "10%",
        RegularCustomer => "5%",
        null => "No customer",
        _ => "Invalid type"
    };
```

}

🔷 15. When to Use Pattern Matching

✓ Type checking
✓ Validation logic
✓ API request handling
✓ Data transformation
✓ Replacing complex if-else or switch